

TOPIC 5 : GREEDY

1. Maximum Coins from Piles

main.py	Visualize	Run	Output
<pre>1 piles = list(map(int, input("Enter piles: ").split())) 2 3 piles.sort() 4 5 n = len(piles) // 3 6 coins = 0 7 i = len(piles) - 2 8 9 for _ in range(n): 10 coins += piles[i] 11 i -= 2 12 13 print("Maximum coins:", coins)</pre>			<pre>Enter piles: 2 4 1 2 7 8 Maximum coins: 9 === Code Execution Successful ===</pre>

2. Minimum Coins to Reach Target

main.py	Visualize	Run	Output
<pre>1 coins = list(map(int, input("Enter coins: ").split())) 2 target = int(input("Enter target: ")) 3 4 coins.sort() 5 6 reach = 0 7 i = 0 8 added = 0 9 10 while reach < target: 11 if i < len(coins) and coins[i] <= reach + 1: 12 reach += coins[i] 13 i += 1 14 else: 15 reach += reach + 1 16 added += 1 17 18 print("Minimum coins to add:", added)</pre>			<pre>Enter coins: 1 4 10 Enter target: 19 Minimum coins to add: 2 === Code Execution Successful ===</pre>

3. Minimum Maximum Working Time

main.py	Visualize	Run	Output
<pre>1 jobs = list(map(int, input("Enter jobs: ").split())) 2 k = int(input("Enter workers: ")) 3 4 jobs.sort(reverse=True) 5 workers = [0] * k 6 7 for job in jobs: 8 i = workers.index(min(workers)) 9 workers[i] += job 10 11 print("Minimum maximum working time:", max(workers))</pre>			<pre>Enter jobs: 1 2 4 7 8 Enter workers: 2 Minimum maximum working time: 11 === Code Execution Successful ===</pre>

4: Greedy – Job Scheduling for Maximum Profit

main.py	Visualize	Run	Output
<pre>1 startTime = list(map(int, input("Enter start times: ").split())) 2 endTime = list(map(int, input("Enter end times: ").split())) 3 profit = list(map(int, input("Enter profits: ").split())) 4 5 jobs = list(zip(startTime, endTime, profit)) 6 jobs.sort(key=lambda x: x[1]) 7 8 n = len(jobs) 9 dp = [0] * n 10 11 dp[0] = jobs[0][2] 12 13 for i in range(1, n): 14 s, e, p = jobs[i] 15 include = p 16 17 for j in range(i - 1, -1, -1): 18 if jobs[j][1] <= s: 19 include += dp[j] 20 break 21</pre>			<pre>Enter start times: 1 2 3 3 Enter end times: 3 4 5 6 Enter profits: 50 10 40 70 Maximum Profit: 120 === Code Execution Successful ===</pre>

5: Greedy – Dijkstra's Shortest Path

main.py	Visualize	Run	Output
<pre>1 INF = float('inf') 2 3 graph = [4 [0, 10, 3, INF, INF], 5 [INF, 0, 1, 2, INF], 6 [INF, 4, 0, 8, 2], 7 [INF, INF, INF, 0, 7], 8 [INF, INF, INF, 9, 0] 9] 10 11 source = 0 12 n = len(graph) 13 14 dist = [INF] * n 15 visited = [False] * n 16 dist[source] = 0 17 18 for _ in range(n): 19 u = -1 20 21 for i in range(n): 22 if not visited[i] and (u == -1 or dist[i] < dist[u]):</pre>			Shortest distances: [0, 7, 3, 9, 5] === Code Execution Successful ===

6: Greedy – Dijkstra's Algorithm Using Edge List

main.py	Visualize	Run	Output
<pre>1 edges = [2 (0, 1, 7), (0, 2, 9), (0, 5, 14), 3 (1, 2, 10), (1, 3, 15), (2, 3, 11), 4 (2, 5, 2), (3, 4, 6), (4, 5, 9) 5] 6 7 n = 6 8 source = 0 9 target = 4 10 11 INF = float('inf') 12 dist = [INF] * n 13 visited = [False] * n 14 dist[source] = 0 15 16 for _ in range(n): 17 u = -1 18 19 for i in range(n): 20 if not visited[i] and (u == -1 or dist[i] < dist[u]): 21 u = i 22</pre>			Shortest path: 26 === Code Execution Successful ===

7. Greedy – Huffman Tree and Huffman Codes

main.py	Visualize	Run	Output
<pre>1 import heapq 2 3 characters = ['a', 'b', 'c', 'd'] 4 frequencies = [5, 9, 12, 13] 5 6 heap = [[frequencies[i], characters[i]] for i in range(len (characters))] 7 heapq.heapify(heap) 8 9 while len(heap) > 1: 10 left = heapq.heappop(heap) 11 right = heapq.heappop(heap) 12 13 new = [left[0] + right[0], left, right] 14 heapq.heappush(heap, new) 15 16 codes = {} 17 18 def generate(node, code=""): 19 if len(node) == 2: 20 codes[node[1]] = code 21 return</pre>			<pre>Huffman Codes: a : 00 b : 01 c : 10 d : 11 === Code Execution Successful ===</pre>

8. Greedy – Huffman Decoding

main.py	Visualize	Run	Output
<pre>1 import heapq 2 3 characters = ['a', 'b', 'c', 'd'] 4 frequencies = [5, 9, 12, 13] 5 encoded = "1101100111110" 6 7 heap = [[frequencies[i], characters[i]] for i in range(len (characters))] 8 heapq.heapify(heap) 9 10 while len(heap) > 1: 11 left = heapq.heappop(heap) 12 right = heapq.heappop(heap) 13 heapq.heappush(heap, [left[0] + right[0], left, right]) 14 15 root = heap[0] 16 decoded = "" 17 node = root 18 19 for bit in encoded: 20 node = node[1] if bit == "0" else node[2] 21</pre>			<pre>Decoded message: dcbdbd === Code Execution Successful ===</pre>

9. Greedy – Maximum Weight Loading

main.py	Visualize	Run	Output
<pre>1 import heapq 2 3 characters = ['a', 'b', 'c', 'd'] 4 frequencies = [5, 9, 12, 13] 5 encoded = "1101100111110" 6 7 heap = [[frequencies[i], characters[i]] for i in range(len 8 (characters))] 9 heapq.heapify(heap) 10 11 while len(heap) > 1: 12 left = heapq.heappop(heap) 13 right = heapq.heappop(heap) 14 heapq.heappush(heap, [left[0] + right[0], left, right]) 15 16 root = heap[0] 17 decoded = "" 18 node = root 19 20 for bit in encoded: 21 node = node[1] if bit == "0" else node[2]</pre>			Decoded message: dbcbdd === Code Execution Successful ===

10. Greedy – Minimum Number of Containers

main.py	Visualize	Run	Output
<pre>1 weights = list(map(int, input("Enter weights: ").split())) 2 capacity = int(input("Enter container capacity: ")) 3 4 weights.sort(reverse=True) 5 6 containers = 1 7 current = 0 8 9 for w in weights: 10 if current + w <= capacity: 11 current += w 12 else: 13 containers += 1 14 current = w 15 16 print("Minimum containers:", containers)</pre>			Enter weights: 5 10 15 20 25 30 35 Enter container capacity: 50 Minimum containers: 4 === Code Execution Successful ===

11. Greedy – Kruskal's Algorithm

main.py	Visualize	Run	Output
<pre>1 def find(parent, x): 2 if parent[x] != x: 3 parent[x] = find(parent, parent[x]) 4 return parent[x] 5 6 def union(parent, a, b): 7 a = find(parent, a) 8 b = find(parent, b) 9 if a != b: 10 parent[b] = a 11 return True 12 return False 13 14 edges = [15 (0, 1, 10), (0, 2, 6), (0, 3, 5), 16 (1, 3, 15), (2, 3, 4) 17] 18 19 n = 4 20 edges.sort(key=lambda x: x[2]) 21 22 parent = list(range(n))</pre>			Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)] Total weight of MST: 19 === Code Execution Successful ===

12. Greedy – Checking Uniqueness of MST

main.py	Visualize	Run	Output
<pre>1 def kruskal(n, edges): 2 parent = list(range(n)) 3 mst = [] 4 total = 0 5 6 def find(x): 7 while parent[x] != x: 8 parent[x] = parent[parent[x]] 9 x = parent[x] 10 return x 11 12 for u, v, w in sorted(edges, key=lambda x: x[2]): 13 a, b = find(u), find(v) 14 15 if a != b: 16 parent[a] = b 17 mst.append((u, v, w)) 18 total += w 19 20 return mst, total 21 22</pre>			Given MST: [(0, 1, 1), (0, 2, 1), (1, 3, 2), (3, 4, 3)] Is the given MST unique? False Another possible MST: [(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)] Total weight of MST: 7 === Code Execution Successful ===